

X-change (Swivastu)

Data Models Documentation

Welcome to the database schema documentation for **X-change (Swivastu)**. This document outlines the core Mongoose schemas required to build a scalable, peer-to-peer local marketplace supporting Barter, Rent, Buy, and Sell operations.

1. User Model (UserSchema)

The core of the platform. It stores basic details, authentication, trust metrics (crucial for P2P), and geospatial location for local searches.

```
const userSchema = new mongoose.Schema({
  firstName: { type: String, required: true },
  lastName: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  passwordHash: { type: String, required: true },
  profileImage: { type: String, default: "default_avatar.png" },
  contactNumber: { type: String, required: true },

  // Location for "Local" Marketplace functionality
  location: {
    type: { type: String, enum: ['Point'], default: 'Point' },
    coordinates: { type: [Number], default: [0, 0] }, // [longitude,
latitude]
    addressText: { type: String } // Pincode, City, Area
  },

  // Trust & Verification metrics
  isVerified: { type: Boolean, default: false },
  trustScore: { type: Number, default: 0 },
  averageRating: { type: Number, default: 0, min: 0, max: 5 },
  totalReviews: { type: Number, default: 0 },

  role: { type: String, enum: ['user', 'admin'], default: 'user' }
}, { timestamps: true });

// NOTE: Index for location-based searching
// userSchema.index({ location: "2dsphere" });
```

2. Item/Product Model (ItemSchema)

Represents the goods listed on the platform. It handles dynamic rules for Bartering, Renting, and Selling.

```
const itemSchema = new mongoose.Schema({
  ownerId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
  title: { type: String, required: true, trim: true },
  description: { type: String, required: true },
  category: { type: String, required: true },
  images: [{ type: String }],
  condition: { type: String, enum: ['New', 'Like New', 'Good', 'Fair',
'Poor'], required: true },

  listingType: [{
    type: String,
    enum: ['Sell', 'Barter', 'Rent', 'Giveaway']
  }],

  // Type specific details
  price: { type: Number },
  rentDetails: {
    pricePerDay: { type: Number },
    securityDeposit: { type: Number },
    maxDurationDays: { type: Number }
  },
  barterPreferences: [{ type: String }], // E.g., ["Bicycle", "Laptop"]

  status: { type: String, enum: ['Available', 'Pending', 'Traded', 'Sold',
'Rented', 'Hidden'], default: 'Available' },

  location: {
    type: { type: String, enum: ['Point'], default: 'Point' },
    coordinates: { type: [Number] }
  }
}, { timestamps: true });
```

3. Transaction / Request Model (TransactionSchema)

The most crucial schema. Handles requests to Buy, Rent, or Barter.

```

const transactionSchema = new mongoose.Schema({
  initiatorId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
  receiverId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
  targetItemId: { type: mongoose.Schema.Types.ObjectId, ref: 'Item',
required: true },
  transactionType: { type: String, enum: ['Buy', 'Rent', 'Barter'], required:
true },

  // IF BARTER: The item the initiator is offering in exchange
  offeredItemId: { type: mongoose.Schema.Types.ObjectId, ref: 'Item' },

  // IF BUY / RENT
  agreedPrice: { type: Number },
  rentDetails: {
    startDate: { type: Date },
    endDate: { type: Date }
  },

  status: {
    type: String,
    enum: ['Pending', 'Accepted', 'Rejected', 'Completed', 'Cancelled',
'Disputed'],
    default: 'Pending'
  },

  // OTP or QR code for physical exchange verification
  completionCode: { type: String }
}, { timestamps: true });

```

4. Chat & Messaging Models

A. Chat Room Schema

```

const chatRoomSchema = new mongoose.Schema({
  participants: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  itemId: { type: mongoose.Schema.Types.ObjectId, ref: 'Item', required:
true },
  lastMessage: { type: String },
  lastMessageAt: { type: Date }
}, { timestamps: true });

```

B. Message Schema

```
const messageSchema = new mongoose.Schema({
  chatRoomId: { type: mongoose.Schema.Types.ObjectId, ref: 'ChatRoom',
    required: true },
  senderId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
    true },
  text: { type: String },
  attachments: [{ type: String }],
  isRead: { type: Boolean, default: false }
}, { timestamps: true });
```

5. Review & Rating Model (ReviewSchema)

For building trust. Users rate each other after a 'Completed' transaction.

```
const reviewSchema = new mongoose.Schema({
  reviewerId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
    true },
  revieweeId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
    true },
  transactionId: { type: mongoose.Schema.Types.ObjectId, ref: 'Transaction',
    required: true },

  rating: { type: Number, required: true, min: 1, max: 5 },
  comment: { type: String, trim: true },
  transactionType: { type: String, enum: ['Buy', 'Rent', 'Barter'] }
}, { timestamps: true });
```

6. Notification Model (NotificationSchema)

Handles system updates, chat alerts, and request statuses.

```
const notificationSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
  type: { type: String, enum: ['Message', 'RequestReceived',
'RequestAccepted', 'RequestRejected', 'System'] },
  title: { type: String, required: true },
  message: { type: String, required: true },

  relatedEntityId: { type: mongoose.Schema.Types.ObjectId },
  onModel: { type: String, enum: ['Transaction', 'ChatRoom', 'Item'] },

  isRead: { type: Boolean, default: false }
}, { timestamps: true });
```



Key Development Tips

- **GeoSpatial Indexing:** Ensure the 2dsphere index is applied to the location field in the User and Item models to use MongoDB's \$near for fast local discovery.
- **References over Embedding:** Using ObjectId keeps the documents lean and makes it easy to populate() the necessary nested data in Express.
- **Barter Automation:** When a Barter transaction's status changes to Completed, set up a Mongoose hook or controller logic to automatically update both the targetItemId and offeredItemId statuses to Traded.